

Homework 3 Programming: Natural Policy Gradient *[50 points]*

CS 1840/Stat 184: Introduction to Reinforcement Learning

(Due November 11 at 11:59 pm ET)

In this problem, we will walk through the Natural Policy Gradient (NPG) algorithm, and also implement it for the CartPole simulation we have worked with previously.

0 Requirements

IMPORTANT: This question uses an older version of `gym`. Please follow these instructions to make sure you have the correct version of all packages.

This question uses Python 3.9 or higher. The required libraries are found in `requirements.txt`. **Do not add any unnecessary imports.** You may not use `scipy` or any other libraries not mentioned above.

You may install the requirements using a package manager of your choice. We recommend `micromamba`, a *much* faster implementation of `conda`. Create a conda environment named `184-hw3` with the required packages using the following terminal commands:

- `micromamba create -n 184-hw3 -c conda-forge -y python=3.9`
- `micromamba activate 184-hw3`
- `pip install -r requirements.txt`

1 Background and Setting

We will use the `CartPole simulator` in OpenAI gym. This environment has two actions: 0 means applying a left push to the cart, and 1 means applying a right push to the cart. It has a four-dimensional state space $\mathcal{S} \subsetneq \mathbb{R}^4$. You will implement the **natural policy gradient** algorithm for learning a stochastic policy.

We will use a **softmax policy**. That is, we encode each state-action pair with the feature mapping $\phi(s, a) \in \mathbb{R}^d$, and then take a softmax:

$$\pi_{\theta}(a \mid s) = \frac{\exp(\theta^{\top} \phi(s, a))}{\sum_{a' \in \mathcal{A}} \exp(\theta^{\top} \phi(s, a'))},$$

where $\theta \in \mathbb{R}^d$ are the parameters. Our goal is of course to find the best θ such that the resulting π_{θ} achieves the largest expected total reward.

The feature mapping ϕ is implemented for you in `utils.py` as `extract_features`. Our particular implementation uses **random Fourier features**. You are not responsible for this material, but in case you are curious, you can find the details in [Random Features for Large-Scale Kernel Machines \(Rahimi and Recht 2007\)](#). ϕ approximates the corresponding features of the radial basis function kernel,

$$\phi(s, a)^\top \phi(s', a') \approx \exp\left(-\frac{1}{2}\|(s, a) - (s', a')\|^2\right).$$

2 Gradient of Policy Log-Likelihood

First derive the expression $\nabla_{\theta} \ln \pi_{\theta}(a | s)$.

Q1 [5 points] Open the file `utils.py`, and implement $\nabla_{\theta} \ln \pi_{\theta}(a | s)$ in `compute_log_softmax_grad`. You should first implement the helper functions `compute_softmax` and `compute_action_distribution`. You should pass the tests in `test.py` for these functions before moving on.

3 Fisher Information Matrix and Policy Gradient

Let us now compute the **Fisher information matrix** for the policy π_{θ} . Recall the definition of the Fisher information matrix:

$$F_{\theta} = \mathbb{E}_{\tau \sim \rho_{\theta}} \left[\sum_{h=0}^{H-1} (\nabla_{\theta} \ln \pi_{\theta}(a_h | s_h)) (\nabla_{\theta} \ln \pi_{\theta}(a_h | s_h))^\top \right] \in \mathbb{R}^{d \times d}.$$

Consider sampling N trajectories $\tau^1, \dots, \tau^N$ from π_{θ} , where $\tau^i = \{s_h^i, a_h^i, r_h^i\}_{h=0}^{H-1}$ with $s_0^i \sim \mu_0$. Then we can approximate F_{θ} using all of the state-action pairs (s, a) from all trajectories:

$$\hat{F}_{\theta} = \frac{1}{N} \sum_{i=1}^N \left[\frac{1}{H} \sum_{h=0}^{H-1} (\nabla_{\theta} \ln \pi_{\theta}(a_h^i | s_h^i)) (\nabla_{\theta} \ln \pi_{\theta}(a_h^i | s_h^i))^\top \right] + \lambda I,$$

where $\lambda \in \mathbb{R}^+$ is a regularization constant used to enforce positive-definiteness.

Q1 [8 points] Go to `train.py` and implement the `sample` function to sample trajectories using a policy. This function should “roll out” (simulate) N trajectories and keep track of the gradients and rewards collected over each trajectory.

Q2 [10 points] Go to `utils.py` and implement $\hat{F}_{\theta}$ in `compute_fisher_matrix`. Note that for our cartpole, there is a termination criteria when we are too far away from the goal. That is, during execution, if the termination criteria is met or we hits the last time step H , the simulator will return `done = True` in the `step` function.

When we generate a trajectory, it is possible that we will just terminate the trajectory early since we might meet the termination criteria before getting to the last time step H . Hence, we will see

that when we collect trajectories, each trajectory might have different lengths. To estimate F_θ , then, you will need to properly average over the trajectory length.

You should pass the tests for `compute_fisher_matrix`. The test solutions were computed using the default lambda value of 10^{-3} . (We don't have tests for `sample`.)

Q3 [9 points] Let $J(\theta)$ denote the objective function:

$$J(\theta) = \mathbb{E} \left[\sum_{h=0}^{H-1} r(s_h, a_h) \middle| s_0 \sim \mu_0, a_h \sim \pi_\theta(\cdot | s_h) \right].$$

Let us implement the policy gradient estimate as

$$\hat{\nabla}_\theta J(\theta) = \frac{1}{N} \sum_{i=1}^N \left[\frac{1}{H} \sum_{h=0}^{H-1} \nabla_\theta \ln \pi_\theta(a_h^i | s_h^i) \left[\left(\sum_{t=h}^{H-1} r_t^i \right) - b \right] \right],$$

where b is a constant baseline $b = \sum_{i=1}^N R(\tau^i)/N$, i.e., the average total reward over a trajectory.

In `utils.py`, implement this PG estimator in `compute_value_gradient`. Again be mindful that each trajectory might have different lengths due to possible early termination. There are tests in `test.py` for this function.

4 Implement the Step Size

Recall that NPG uses the following update:

$$\theta' := \theta + \eta \hat{F}_\theta^{-1} \hat{\nabla}_\theta J(\theta),$$

where θ' denotes the next iterate of the parameters. We need to specify the step size η . To do this, we'll draw from the **trust region** interpretation of NPG; that is, we want to ensure that the KL divergence between the trajectory distributions before and after is not that big.

Recall that the KL divergence $KL(\rho^{\pi_\theta} \parallel \rho^{\pi_{\theta'}})$ can be approximated by the Fisher information matrix as follows (ignoring constant factors):

$$KL(\rho^{\pi_\theta} \parallel \rho^{\pi_{\theta'}}) \approx (\theta - \theta')^\top F_\theta (\theta - \theta').$$

As explained in lecture, instead of using the learning rate as the hyper-parameter for NPG, we instead use the trust region size (which has a more transparent interpretation) as a hyper-parameter; that is, we set δ such that

$$KL(\rho^{\pi_\theta} \parallel \rho^{\pi_{\theta'}}) \approx (\theta - \theta')^\top \hat{F}_\theta (\theta - \theta') \leq \delta.$$

Since $\theta' - \theta = \eta \hat{F}_\theta^{-1} \hat{\nabla}_\theta J(\theta)$ by definition, we have

$$\eta^2 (\hat{\nabla}_\theta J(\theta))^\top \hat{F}_\theta^{-1} \hat{\nabla}_\theta J(\theta) \leq \delta.$$

Solving, we get $\eta \leq \sqrt{\frac{\delta}{(\hat{\nabla}_{\theta} J(\theta))^{\top} \hat{F}_{\theta}^{-1} \hat{\nabla}_{\theta} J(\theta)}}$. We will just set $\eta = \sqrt{\frac{\delta}{(\hat{\nabla}_{\theta} J(\theta))^{\top} \hat{F}_{\theta}^{-1} \hat{\nabla}_{\theta} J(\theta)}}$, i.e., be aggressive on setting the learning rate subject to the trust region constraint. To ensure numerical stability when the denominator is close to zero, we add $\epsilon = 10^{-6}$ to the denominator, resulting in the expression

$$\eta = \sqrt{\frac{\delta}{(\hat{\nabla}_{\theta} J(\theta))^{\top} \hat{F}_{\theta}^{-1} \hat{\nabla}_{\theta} J(\theta) + \epsilon}}.$$

Q1 [5 points] Go to `utils.py` and implement the step size computation in `compute_eta`. Check your implementation with the tests in `test.py`.

5 Putting Everything Together

Q1 [10 points] Now we can put all the pieces together. Go to `train.py` to implement the main framework in `train`. In each iteration of NPG, `train` should do the following steps:

1. Collect samples by rolling out N trajectories with the current policy using the `sample` function.
2. Compute the (estimated) Fisher Information matrix $\hat{F}_{\theta}$ using the gradients from the previous steps. Use the default value of λ .
3. Compute the step size for this NPG step.
4. Update the model parameters, θ , by taking an NPG step.

In addition to the above 4 steps, keep track of the average episode reward in each iteration of the algorithm. The output of `train` should be the final model parameters and a list containing the average episode rewards for each step of NPG.

Q2 [3 points] Run the above algorithm with the parameters $T = 20$, $\delta = 10^{-2}$, $\lambda = 10^{-3}$, and $N = 100$. Report the performance of each π_{θ_t} for $t = 0$ to 19 by recording the average reward received from each episode, and plot the performance curve. You should be able to do this by simply running `train.py` from the terminal. Save your plot to `average_reward.png`.

Your algorithm should achieve average reward of over 190 in about 15 steps if implemented correctly.

6 Submission

Please submit a .zip file with directory tree like the following:

```
YOUR_HUID/
├── average_reward.png
├── README.md
├── requirements.txt
└── train.py
```

```
|  
└─ test.py  
└─ test_info.pkl  
└─ utils.py
```